

Moving the prior onto the code

A Gaussian-mixture latent for DUALVAE — math and intuition

Abstract

This note explains, from scratch, the idea of “summing the codes into the mean before the reparameterization trick” — and why it only helps if it comes with a matching change to the *prior*. The upshot: make the discrete code the *center* of the Gaussian prior for that location. The continuous branch then encodes a small *residual* around the code, the marginal latent becomes a genuine *Gaussian mixture* with the codewords as component means, and (unlike the current model) the mixture can be well separated.

1 The current model, in symbols

At each latent location the model builds

$$z = z_{\text{vq}} + z_{\text{cont}}, \quad z_{\text{cont}} = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (1)$$

where $z_{\text{vq}} = e_k$ is the chosen codeword (the discrete branch) and z_{cont} is the continuous (VAE) branch, produced by the reparameterization trick from a predicted mean μ and standard deviation σ . Training maximizes a reconstruction term minus a KL regularizer

$$\mathcal{L} = \underbrace{\mathbb{E} \|x - \hat{x}(z)\|^2}_{\text{reconstruction}} + \beta \underbrace{\text{KL}(q(z_{\text{cont}} | x) \| p(z_{\text{cont}}))}_{\text{regularizer}}, \quad p(z_{\text{cont}}) = \mathcal{N}(0, I). \quad (2)$$

Why this gives *no* mixture. The KL in (2) pulls the continuous posterior $\mathcal{N}(\mu, \sigma^2)$ toward $\mathcal{N}(0, I)$, i.e. it pushes $\mu \rightarrow 0$ — *independently of the code*. The code z_{vq} is just added afterwards and is never tied to the continuous distribution. So the continuous branch is free to roam a full unit-variance Gaussian around the origin, and empirically its spread (~ 1.0) dwarfs the spacing between codewords (~ 0.1). The codewords are then tiny perturbations inside one big blob: the mixture is there in principle ($z = e_k + \text{noise}$) but completely *unseparated*.

2 The idea: condition the prior on the code

Split the continuous branch into a *residual* Δ around the code:

$$z = z_{\text{vq}} + \Delta, \quad \Delta = \mu_{\Delta} + \sigma \odot \varepsilon, \quad q(\Delta | x) = \mathcal{N}(\mu_{\Delta}, \sigma^2). \quad (3)$$

Now change the prior so that, *given* the code k , the latent is expected to sit *near the codeword* rather than near the origin:

$$\boxed{p(z | k) = \mathcal{N}(e_k, s_k^2 I)} \quad \Longleftrightarrow \quad p(\Delta | k) = \mathcal{N}(0, s_k^2 I). \quad (4)$$

That is, the **codeword becomes the prior mean** and s_k is a (small) per-code spread. The regularizer in (2) becomes a KL between the residual posterior and this code-centered prior, which has a clean closed form (per dimension):

$$\text{KL}(\mathcal{N}(\mu_\Delta, \sigma^2) \parallel \mathcal{N}(0, s_k^2)) = \log \frac{s_k}{\sigma} + \frac{\sigma^2 + \mu_\Delta^2}{2s_k^2} - \frac{1}{2}. \quad (5)$$

The important term is $\mu_\Delta^2/(2s_k^2)$: it penalizes the residual *mean*, so training pushes $\mu_\Delta \rightarrow 0$, i.e. it pushes z toward the codeword e_k . The code now *anchors* the latent instead of being an afterthought.

3 The marginal latent is a Gaussian mixture

Averaging (4) over how often each code is used (π_k = code-usage frequency) gives the marginal prior

$$p(z) = \sum_k \pi_k \mathcal{N}(e_k, s_k^2 I) \quad (6)$$

— a **Gaussian mixture model** whose *component means are the codewords*, whose *weights are the code frequencies*, and whose *component widths* are the s_k . This is exactly the object we were testing for; the change in (4) is what makes it real instead of nominal.

4 Intuition in one paragraph

Think of each codeword as a labelled *bin* in latent space. The discrete branch says “this patch belongs to bin k ” (the anchor e_k); the continuous branch adds a small *wiggle* Δ inside that bin (the within-bin detail). The code-centered prior (4) tells the wiggle to stay *small and centered on the bin*. If the bins are farther apart than the wiggle is wide, the bins don’t overlap and you get a clean mixture:

$$\text{well separated} \iff s_k \ll \underbrace{\min_{j \neq k} \|e_j - e_k\|}_{\text{codeword spacing}}. \quad (7)$$

In the current model the wiggle (~ 1.0) is much wider than the spacing (~ 0.1), so the bins are smeared into one blob. Condition (7) is precisely the “keep σ below the codeword spacing” rule — now enforced by the prior instead of hoped for.

5 Why “sum before reparameterization” *alone* does nothing

It is tempting to think the fix is just to move the $+z_{\text{vq}}$ to before the noise:

$$\text{after (current): } z = z_{\text{vq}} + (\mu + \sigma \odot \varepsilon), \quad (8)$$

$$\text{before: } z = (\mu + z_{\text{vq}}) + \sigma \odot \varepsilon. \quad (9)$$

These produce the *identical* $z = z_{\text{vq}} + \mu + \sigma \odot \varepsilon$ — same value, same gradients, same noise. If the KL still regularizes μ toward $\mathcal{N}(0, \cdot)$, **nothing changes**. The re-ordering only becomes meaningful when it is paired with the *prior* move in (4): regularize the combined mean $(\mu + z_{\text{vq}})$ toward the code-centered prior $\mathcal{N}(e_k, s_k^2)$, which — by (5) — reduces to regularizing the *residual* μ toward 0. (Beware the wrong version: regularizing $(\mu + z_{\text{vq}})$ toward $\mathcal{N}(0, I)$ would penalize $\|z_{\text{vq}} + \mu\|^2$ and *fight* the codes, trying to cancel them.)

6 How this is wired in the code

This is already available through three switches (base it on the Run B config, $\beta=0.1$):

Flag	Effect
<code>residual_continuous: true</code>	continuous branch encodes $\Delta = z_e - z_q$, so $z = e_k + \Delta$ literally
<code>component_prior: true</code>	KL prior on Δ is $\mathcal{N}(0, s_k^2)$ (Eq. 4), not $\mathcal{N}(0, I)$
<code>use_ema_codebook: true</code>	required by <code>component_prior</code> : s_k^2 is the EMA within-code variance

Here s_k^2 is estimated online as the average squared residual assigned to code k ($s_k^2 = v_k/N_k$), clamped to `[sigma2_floor, sigma2_ceil]`. The floor is an anti-collapse valve (it stops the “tighter prior $\rightarrow$ smaller residual $\rightarrow$ tighter prior” ratchet); the ceiling stops the model inflating residuals to loosen its own prior. The provided config `configs/dualvae_code_centered_prior.yaml` sets all three.

7 What success looks like (and how to check it)

Re-run `tools/dualvae_latent_analysis.py` and `tools/branch_contribution.py` on the new checkpoint and watch:

- **Separability ratio** and **2-mode separation** should rise well above the Run B values (0.68, 2.09) — ideally separability > 1 , i.e. condition (7).
- **s_k vs codeword spacing**: the per-code std should sit *below* the spacing (watch `Codebook/Sigma2 *` in wandb; if `Sigma2 At Floor Frac` is high, lower `sigma2_floor`).
- **Code-info share** (`full-code_mean`) should stay high, and the code-anchored held-out log-likelihood should now *beat* a single Gaussian.
- **Reconstruction** (PSNR/SSIM/LPIPS) should stay close to Run B — the goal is to gain mixture structure without losing much fidelity. Tune β (mixture pressure) and `sigma2_floor` (component tightness) if it costs too much.

This is the standard conditional/learned-prior recipe (cf. VQ-VAE-2 hierarchical priors and GMM-VAE), specialized so the mixture components are exactly your codebook entries.